Part 2: Hashing with Chaining

Stephanie Peguero

1. Implementation

A Hash Table was implemented using chaining to resolve collisions. In this approach, the hash table consists of an array where each index (or "bucket") holds a list. When multiple keys hash to the same index, their key-value pairs are simply appended to the list at that index.

To determine the correct bucket for a given key, the implementation utilizes Python's built-in hash() function combined with the modulo operator: index = hash(key) % capacity. Python's hash() function is highly optimized and distributes data evenly, functioning as a suitable, efficient hash function to minimize collisions across the available capacity (Python Software Foundation, 2026).

The implementation supports the three core operations efficiently:

- **Insert:** Calculates the index and iterates through the bucket's list. If the key already exists, its value is updated. If not, the new (key, value) tuple is appended to the end of the list.
- **Search:** Calculates the index and iterates through the list at that bucket. If the key matches an existing tuple, the associated value is returned. If the loop completes without a match, it returns None.
- **Delete:** Calculates the index and uses the enumerate() function to track the list index while iterating. If the key is found, it is removed using .pop(i), returning True. If the key does not exist, it returns False.

2. Analysis

To analyze the efficiency of this hash table, the assumption of simple uniform hashing is utilized. This assumes that any given key is equally likely to hash into any of the available slots, independently of where other keys have hashed (Miller & Ranum, 2011).

Under this assumption, the performance of the table is dictated by the load factor, denoted mathematically as α. The load factor is defined as the ratio of the total number of elements stored in the table (n) to the total number of available slots/buckets (m):

$$\alpha = \frac{n}{m}$$

Because of simple uniform hashing, the average length of any given chain in the table is exactly α (Miller & Ranum, 2011).

Consequently, the expected time complexity for Search, Insert, and Delete operations is $O(1 + \alpha)$.

- The $O(1)$ component represents the constant time required to compute the hash function and access the specific bucket array index.
- The $O(\alpha)$ component represents the expected time required to linearly scan the chain at that bucket to find, update, or delete the specific key.

Impact of the Load Factor: If the number of items (n) remains proportional to the number of slots (m), then α is a small constant, and all operations run in $O(1)$ expected time. However, if n grows significantly larger than m, the load factor α increases proportionally. The chains become extremely long, and the time complexity degrades toward $O(n)$, essentially reducing the hash table to an unsorted linked list.

Strategies for Minimizing Collisions: To maintain optimal $O(1)$ performance, a low load factor must be maintained. The primary strategy for achieving this is dynamic resizing (Miller & Ranum, 2011).

When the hash table detects that the load factor has exceeded a certain threshold (commonly $\alpha > 0.7$), the table must be resized. This typically involves allocating a new underlying array with double the capacity ($2m$).

Because the hash function relies on the table's capacity (hash(key) % capacity), changing the capacity fundamentally alters where items belong. The old lists cannot simply be copied over. Instead, rehashing must be performed, iterating through every item in the old table, recalculating its new index modulo the new capacity, and inserting it into the new table. While rehashing takes $O(n)$ time, it happens infrequently enough that the amortized cost of insertion remains $O(1)$, ensuring the hash table remains highly efficient as data scales (Miller & Ranum, 2011).

References

Miller, B. N., & Ranum, D. L. (2011). *Problem solving with algorithms and data structures using Python* (2nd ed.). Franklin, Beedle & Associates.

Python Software Foundation. (n.d.). *sys — System-specific parameters and functions*. Python 3 Documentation. Retrieved from https://docs.python.org/3/library/sys.html